Metrics	C++
Programming Paradigm	Object-Oriented Programming (OOP)
Origin	Based on C language
Developer	Bjarne Stroustrup in 1979
Translator	Compiler only
Platform Dependency	Platform Dependent
Code execution	Direct
Approach	Bottom-up approach
File generation	.exe files
Pre-processor directives	Supported (#header, #define)
keywords	Supports 63 keywords
Datatypes (union, structure)	Supported
Inheritance	Supported
Overloading	Support Function overloading (Polymorphism)
Pointers	Supported
Allocation	Use new, delete
Exception Handling	Supported
Templates	Supported
Destructors	Supported

Bottom Up - Top Down

↳ Bottom
smaller probs solved
· OOP Paradigm C++, Java, Python etc
· used for testing mods
· doesn't use redundancy
· composition approach
· data encapsulation + hiding used.
↳ Top
· used by structured langs C, COBOL, FORTRAN
· used with mod/debug code
· no mod communication
· uses info redundancy
· decomp approach
· can/cannot use data encap/hiding

Namespaces

↳ Group of "named" entities
- constants, variables, functions, structures, classes, nesting capabilities
↳ 3 use methods
- fully qualified, local import, global import

```
full qual: std::cout<<'Hi'<<std::endl;
local import:
    int main()
    { using namespace std:
      cout<< "Hello" <<endl; }
global import:
  using namespace std;
  int main(){
     cout<< "Hello"<<endl;}
```

C++ String

↳ Built in C++ class
↳ replaces C char*
↳ hold const strings, var strings
↳ supports range of operators

C++ Constness

↳ make entity immutable, reducing risk of accidental change.
↳ Annotation added on declarations of vars, of functions' args

C++ numeric types

↳ same as C99
· char[1], short[2], int[4], long[4,8], long long[8]
· float[4], double[8]
· new → bool[1]

Memory Models

↳ three pools
- static
- stack
- heap
↳ Each differs in lifetime & allocation/deallocation policy

Static Mem

↳ Memory allocated at compile time
↳ read/write
↳ one copy not dynamic
↳ use 'static' keyword
↳ lifetime = entire prog exec

Stack

↳ Mem allocated for local vars
↳ allocate at funct enter/ deallocate at exit
↳ each recursive call gets own set of local vars

Heap

↳ mem from manual "on-the-fly" allocations
↳ 2 simple APIs: new and delete
↳ coder control allo/dealloc
↳ lifetime = till freed

Pointers & Constness

```
int x = 10
const int* px = &x;
*px  ← x int is const

int x=10;
int* const px=&x
*px=20
px=px+1  ← x address is const
```

Spiral Rule To Type Read

↳ start at unknown element and work in spiral to replace with english

str is an array 10 of pointers to char

fp is a pointer to a function passing an int and a pointer to float returning a pointer to a char

Casting

casting operator are used for type casting/converting from one data type to another
↳ types
- static_cast
- dynamic_cast
- const_cast
- reinterpret_cast

C++ Pointers

· Get "address" of something
↳ dereference an address to get what's stored there
↳ compute address of something

& and * operators

& takes address of
* Follow the pointer to

```
int x = 10
int* px = &x
*px = 20
(px not==20)
```

Exception Reporting

↳ throw <expr>
· a statement
· with an expression argument
· val of expression is what is thrown
↳ semantics:
- normal control flow path is altered
- flow moved to catch site
- no catch site return to callee (if main reached and nothing os kills prog

Exception Handling

```
try{
<block0>
}catch(<decl1>){
<block1>
}
```

try running block 0 if fail look for fail case corresponding catch block. run first found corresponding

STL Containers

↳ containers/container classes store objects and data
↳ sequence containers: implement data structures accessed in sequential manner
vector, list, deque, arrays, forward - list
↳ container adaptors
↳ provide a different interface for sequence
↳ Queue, priority-queue, stack
↳ associative containers
↳ quickly sortable data structures O(log n) ↳ set, multiset, map, multimap
↳ unordered associative
↳ quick searchable unordered
unordered version of all associative

References with const

↳ same semantics as pointers
↳ use spiral rule to read type still
↳ enforces read only

```
int sumOf(const string&
a) { int ttl=0;
for(int i=0; i<a.size()
; i++){ ttl +=a[i] ret ttl;}
```

References

↳ only in C++
↳ same purpose as pointers
→ advantages
= cleaner syntax
stricter semantics
↳ can mix with pointers + constness

OCD

Protection → state + strong cohesion
Contract → Behavior

Function Overriding

↳ redefinition of base class function in the derived class, will keep signature
↳ eg of runtime polymorphism
↳ overridden functions are in different scopes
↳ needed when derived class function has to do some different job than the base class function
can only be overriden once by each child class
↳ this use inheritance

Object Life-time (Birth)

↳ uses constructors
- execute a method when creating an object
there are many different constructors that can be used

Stack

↳ LIFO working container
↳ uses encapsulated objects of vector of deque or list as underlying container, with specific set of member funcs for element access
↳ stack.push(i) to add i
↳ stack.top() for 1st elem
↳ stack.pop() to remove 1st

Vector

vector provides
↳ a sequence
↳ Dynamic sizing
↳ add/remove anywhere
↳ Rand access
↳ Bound safety

Iterators

↳ abstractions of indexes into containers
↳ iterator can be made to point to:
- beginning
- right past
↳ can be moved
- forward
- Backward
- to a rank

Struct vs. Class

↳ The two constructs are the same in C++ except struct defaults to public and class defaults to private

Function Overloading

↳ multiple definitions of the function by changing signature
↳ eg of compile time polymorphism
↳ funct signatures must be different
↳ overloaded functions must be in the same scope
↳ overloading is used when same funct must behave different based of parameters passed to it.
↳ has ability to load multiple times.
↳ no inheritance

Protection

↳ Public: everyone can access class members can be accessed by anything anywhere in program
↳ private: can only be accessed by member functions
↳ protected: only members and subclasses can access all info

copy vs Move Codesnips

```
copy:
 Student & op=(const Student&s){
   if(this==&s) return *this;
   _name = s._name;
   _mt = s._final;
   _homework = s._homeworks;
   return *this;}
move:
 Student& operator=(Student&&s){
 _name =std::move(s._name);
 _mt=s._mt;
 _final=s._final;
 _homeworks=std::move(s._hws);
 return *this;}
```

Object Lifetime (Death)

↳ Death with destructors
↳ release resource held by the instance
↳ only one destructor
↳ can use virtual keyword to handle polymorphic destruction

Constructor types

↳ Default: when no args are provided
↳ custom: called with arguments to set up the specific instance
↳ copy: initializes an object using another object of same class. Source objects will have its member field copied to new object
↳ move: source object that is passed to the constructor has it's member fields moved into the new object

Abstract Class

↳ state + behavior
↳ no implementation (could be partial)
↳ corollary
- one never instantiates an abstract
- one only subclasses an abstract
↳ how to use an abstract class
- use inheritance to claim u support the contract
- Provide an implementation in the sub-class for "pure" methods

```
Class AStudent{
Public:
 Virtual void read(
 std::istream& is)
 =0; };
```

Inheritance

↳ promote code reuse
↳ supports type refinements
↳ sub-typing through sub-classing

Inheritance + Privacy

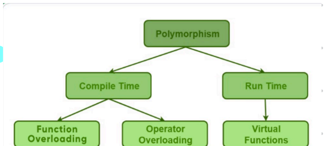
↳ when you inherit you can alter the privacy of what u inherit
A public inherits B
A = everyone knows
A protected inherits B
A = only A + B + A-subs know
A private inherits B
A = only A+B know

Inheritance + Overriding

↳ when a inherits from B
- has all B attributes
- has all B methods
- can add methods
- can update methods (override)
- need to achieve Runtime polymorphism through dynamic binding of overriden methods

C++ Polymorphism

↳ compile-time polymorphism
- funt/operator overloading
↳ Runtime
- object ability to respond to messages based on dynamic type insted of compile-time type
→ funt overriding
- virtual funct
- virtual keyword switches from static binding to dynamic binding

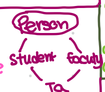


Multiple Inheritance

↳ C++ feature where a class can inherit from more than one class constructors called in order of inheritance destructors in opposite order

Diamond Prob

↳ occurs when two superclasses have shared base



Object-Oriented (OO) Design

↳ abstraction, encapsulation, inheritance, polymorphism
↳ objects package data and procedures (method) that operate on the data
- client → send a request
→ server performs an operation
- Request is the only way to change object's internal data: encapsulation
↳ OO Design
- Decompose into set of manageable and interacting objects
- encapsulation, granularity, flexibility, performance, evolution, reusability, and so on

Object Interface and Interface

↳ signature of operation/method: name, parameters, and return value
↳ object interface: the set of all signatures of its operations
↳ type: name of an interface
- object a has type A → a accepts all request in A
- one object w multiple types (multiple inheritance)
- widely different object w same type
- subtype: inherit another interface
Inheritance: A way of building software incrementally
Polymorphism: a variable referring to type A can refer to an object of type B if B is a subclass of A through dynamic binding

Abstraction

↳ hiding complex implementation details and showing only essential object features achieved w abstract classes and inteface

Encapsulation

↳ Budling data and methods that operate on that data within a class, while restricting direct access using access specifiers

Inheritance

↳ Creating a new class from an existing class, allowing reuse of code and extension of functionality

Polymorphism

↳ the ability for functions or objects to behave different based on their context achieved in C++ via funct overloading and virtual functions for runtime polymorphism.

class inheritance

↳ class inheritance defines an objects implementation in terms of another objects implementation

interface inheritance

subtyping for short describes when a object can be used in place of another

Association: typically named using a verb or verb phrase reflecting real world prob domain
Aggregation: type of association repesenting a part of relationship C2 associated w C1 C1 & C2 have seperate lifetimes
Composition: parts are destroyed when whole is destroyed P1 and P2 destroyed together.
Dependancy: object of one class uses object of another class in the code of a method. If object not stored in any field then this is dependancy

class

↳ define how object is implemented

type

↳ refers to interface/ set of requests it can respond to
- object can has many types

Program to interface not implementation
↳ promotes flexibility, maintainability and testability through decoupling
Principle 1

Design Patterns

↳ names, abstracts, and identifies key aspects of common design structure that make it useful for creating a reusi OOD.

Inheritance: white-box reuse

↳ white-box refers to visibility with inheritance, the internals of parent classes are often visible to subclasses

Composition Black-Box reuse

↳ requires object's fully composed form with highly developed interfaces, this is black-box because no internal deets are visible

We favor composition over inheritance
Principle 2

OO vs procedural design

OO Pros: flexi, reusi, easy to maintain through decoupling
OO cons: complex, less efficient
Procedural: better for small software changes

Purpose Criterion reflect what a pattern does:
↳ creation pattern concerns the process of object creation ↳ Behavioral: class ways classes/objects interact & distribute responsibilities
↳ Structural patterns: deal with compos. of classes/objects

Design For Change

↳ algorithic dependance
↳ tight coupling
↳ subclassing
↳ inability to alter classes conveniently
↳ create object by specifying a particular class
↳ specify operation
↳ dependence on soft/hardware platform
↳ Dependance on object representation and implem.

Creational

↳ uses inheritance/composition

↳ singleton

typically we have multiple instantiated objects from 1 class
↳ this is when we have only one that can be accessed globally.

```
class Singleton {
public:
    static Singleton* Instance();
protected:
    Singleton();
private:
    static Singleton* _instance;
};
```

```
Singleton* Singleton::Instance () {
    if (_instance == 0) {
        _instance = new Singleton;
    }
    return _instance;
}
```

Factory Method (Virtual Constructor)

↳ Interface for creating an object, but let subclasses decide which class to instantiate

```
class Transport {
public:
    // Virtual method to be implemented by concrete products
    virtual void deliver() = 0;
    virtual ~Transport() = default;
};

// Concrete Product: Car
class Car : public Transport {
public:
    void deliver() override {
        std::cout << "Delivering by Car\n";
    }
};
```

Abstract factory

↳ provide interface for creating families of related/dependant objects without specifying concrete classes

```
// ----- Abstract Products -----

class Button {
public:
    virtual void render() = 0;
    virtual ~Button() = default;
};
// ----- Concrete Products: Windows -----

class WindowsButton : public Button {
public:
    void render() override {
        std::cout << "Rendering Windows Button\n";
    }
};
```

Builder

↳ separate the construction of a complex object from it's representation, so same construction process can create different representations

```
// Product
class Meal {
public:
    void setMain(const std::string& main) { mainDish = main; }
    void setDrink(const std::string& d) { drink = d; }
    void setDessert(const std::string& d) { dessert = d; }
    void show() const {
        std::cout << "Main: " << mainDish << ", Drink: " << drink << ", Dessert: " << dessert << "\n";
    }
private:
    std::string mainDish, drink, dessert;
};

// Abstract Builder
class MealBuilder {
public:
    virtual void buildMain() = 0;
    virtual void buildDrink() = 0;
    virtual void buildDessert() = 0;
    virtual std::unique_ptr<Meal> getMeal() = 0;
    virtual ~MealBuilder() = default;
};

// Concrete Builder: Italian
class ItalianBuilder : public MealBuilder {
    std::unique_ptr<Meal> meal = std::make_unique<Meal>();
public:
    void buildMain() override { meal->setMain("Lasagna"); }
    void buildDrink() override { meal->setDrink("Wine"); }
    void buildDessert() override { meal->setDessert("Tiramisu"); }
    std::unique_ptr<Meal> getMeal() override { return std::move(meal); }
};
```

scope criterion: whether pattern applies to classes or object
↳ **class**: deal w. connection btwn classes & subs
- static (fixed at compile)

↳ **object patterns**: deal w. object connections
- can be changed at run-time (more dynamic

Structural Design Patterns

how classes are composed to form larger structures

Composite pattern

↳ objects contain other objects
↳ treats comp. object the same as atomic

```
class Node {
public:
    virtual void show(int indent = 0) const = 0;
    virtual ~Node() = default;
};

class File : public Node {
    std::string name;
public:
    File(std::string n) : name(std::move(n)) {}
    void show(int indent = 0) const override {
        std::cout << std::string(indent, ' ') << "File: " << name << "\n";
    }
};

class Folder : public Node {
    std::string name;
    std::vector<std::unique_ptr<Node>> items;
public:
    Folder(std::string n) : name(std::move(n)) {}
    void add(std::unique_ptr<Node> n) { items.push_back(std::move(n)); }
    void show(int indent = 0) const override {
        std::cout << std::string(indent, ' ') << "Folder: " << name << "\n";
        for (const auto& item : items) item->show(indent + 2);
    }
};
```

Bridge Pattern

↳ Decouple an abstraction from its implementation so that the two can vary independently

```
// Implementor
class DrawAPI {
public:
    virtual void draw() = 0;
};

// Concrete Implementor
class RedAPI : public DrawAPI {
public:
    void draw() override { std::cout << "Drawing Red Shape\n"; }
};

// Abstraction
class Shape {
protected:
    DrawAPI* api;
public:
    Shape(DrawAPI* a) : api(a) {}
    virtual void draw() = 0;
};
```

Proxy Pattern

↳ provide surrogate or placeholder for another object to control access to it can defer cost of creation and initialization until we actually need to use

```
#include <iostream>

class Subject {
public:
    virtual void request() = 0;
};

class RealSubject : public Subject {
public:
    void request() override { std::cout << "Handling request\n"; }
};

class Proxy : public Subject {
    RealSubject* realSubject;
public:
    Proxy(RealSubject* r) : realSubject(r) {}
    void request() override { std::cout << "Checking permissions\n"; realSubject->request(); }
};
```

Decorator

↳ attach additional responsibilities to an object dynamically. flexible alternative to subclassing for extending functionality

```
// Base class defining the coffee interface
class Coffee {
public:
    virtual double cost() const = 0;  // Pure virtual function to calculate cost
};

// Concrete component: Simple coffee
class SimpleCoffee : public Coffee {
public:
    double cost() const override { return 5.0; }  // Base cost of simple coffee
};

// Base decorator class
class CoffeeDecorator : public Coffee {
protected:
    Coffee* coffee;  // Holds a reference to a Coffee object
public:
    CoffeeDecorator(Coffee* c) : coffee(c) {}  // Constructor accepts a Coffee object
    double cost() const override { return coffee->cost(); }  // Delegate cost calculation
};

// Concrete decorator: Adds milk to the coffee
class MilkDecorator : public CoffeeDecorator {
public:
    MilkDecorator(Coffee* c) : CoffeeDecorator(c) {}  // Pass coffee object to base decor
    double cost() const override { return coffee->cost() + 1.5; }  // Add milk cost
};
```

Behavioral

↳ algorithms and assignment of responsibilities between objects (also patterns of communication between object/classes

Command

↳ encapsulate a request as an object (turn request into an object) letting you parameterize clients with different requests, queue of log requests, and support undoable operations

```
// Command interface with a single execute method
class Command {
public:
    virtual void execute() = 0;
};

// Receiver: contains the real business logic
class Light {
public:
    void on() { std::cout << "Light ON\n"; }
};

// Concrete Command: binds a receiver (Light) to an action
class LightOn : public Command {
    Light* light;
public:
    LightOn(Light* l) : light(l) {}
    void execute() override { light->on(); }  // Delegates action to receiver
};
```

Observer

↳ Define a one to many dependency between objects so when the object changes state, all dependents are notified and updated automatically

```
// Observer interface
struct Observer { virtual void update(int) = 0; };
// Subject (Observable)
class Weather {
    std::vector<Observer*> obs; int temp = 0;
public:
    void add(Observer* o) { obs.push_back(o); }
    void set(int t) {
        temp = t;
        for (auto o : obs) o->update(temp); }}; // notify
// Concrete Observer
struct Display : Observer {
    void update(int t) override { std::cout << t << "\n";
};
```

Strategy

↳ Define a family of algorithms encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary indep. from clients that use it

```
// Strategy interface
struct Strategy { virtual void execute() = 0; };
// Concrete strategies
struct Fast : Strategy { void execute() override { std::cout << "Fast\n"; } };
struct Slow : Strategy { void execute() override { std::cout << "Slow\n"; } };
// Context: uses a Strategy
class Context {
    Strategy* s;
public:
    Context(Strategy* strat) : s(strat) {}
    void set(Strategy* strat) { s = strat; }
    void run() { s->execute(); }
```

State

↳ allow an object to alter behavior when its internal state changes
↳ useful when object depends on its state, and it must change its behavior at run-time depending on that state.

```
// State interface
struct State { virtual void handle() = 0; };
// Concrete states
struct On : State { void handle() override { std::cout << "State: ON\n"; } };
struct Off : State { void handle() override { std::cout << "State: OFF\n"; } };
```

```
// Context: behavior changes based on current state
class Switch {
    State* state;
public:
    Switch(State* s) : state(s) {}
    void set(State* s) { state = s; }  // Change state
    void press() { state->handle(); }  // Delegate behavior
};
```

```
// Wildcard player: qualify for any tournament
class ECSportsPlayerWildcard : public ECSportsPlayerDec
{
public:
    ECSportsPlayerWildcard(ECSportsPlayer &playerOrig) : ECSportsPlayerDec(playerOrig) {}
    virtual bool PositivePlay(ECTournament &tr) const { return true; }
};
```

```
class ECSportsPlayerDec : public ECSportsPlayer
{
public:
    ECSportsPlayerDec(ECSportsPlayer &player)
     : player(player) {}
    virtual bool PositivePlay(ECTournament &tr)
        const { return player.PositivePlay(tr); }
    virtual bool NegPlay(ECTournament &tr)
        const { return player.NegPlay(tr); }
private:
    ECSportsPlayer &player; };
```